

Instituto Tecnológico de Tepic



Ing. Sistemas Computacionales

Investigación y cuadro sinóptico

Programación Web.

Unidad 4. Programación del lado del servidor

Landeros Villarruel Luis Angel - 23400569

JavaScript: conceptos fundamentales del lenguaje

1. Características básicas del lenguaje

JavaScript es un lenguaje de programación interpretado, aunque los motores modernos (como V8, usado en Chrome y Node.js) lo compilan "al vuelo" (Just-In-Time compilation) para mejorar el rendimiento en tiempo de ejecución, en lugar de compilarse por completo antes de ejecutarse como ocurre en lenguajes como C o Java.

Es de tipado dinámico y débil: las variables no requieren declarar un tipo de dato al crearse, y ese tipo puede cambiar libremente durante la ejecución del programa (una variable puede contener un número y luego una cadena de texto sin generar error). Esto brinda flexibilidad, pero también puede provocar errores difíciles de detectar si no se maneja con cuidado.

Es multiparadigma, ya que permite programar de forma imperativa (instrucciones paso a paso), orientada a objetos (basada en prototipos en lugar de clases puras, aunque desde ES6 se incluye la sintaxis class como "azúcar sintáctico") y funcional (funciones como ciudadanos de primera clase, se pueden almacenar en variables, pasar como argumentos y retornar desde otras funciones).

Se ejecuta principalmente en el navegador (front-end), donde interactúa con el DOM y eventos del usuario, pero también en el servidor mediante entornos como Node.js, lo que permite crear aplicaciones completas con un mismo lenguaje. Su modelo de ejecución es de un solo hilo (single-thread), lo que significa que solo puede ejecutar una tarea a la vez; para manejar operaciones que tardan (como peticiones a un servidor) se apoya en un Event Loop y una cola de tareas, permitiendo simular concurrencia sin bloquear la interfaz.

2. Estructuras de control

Condicionales: la estructura if evalúa una condición booleana y ejecuta un bloque de código solo si esta es verdadera; else if permite encadenar condiciones adicionales, y else define un bloque alternativo cuando ninguna condición se cumple. La estructura switch es útil cuando se necesita comparar una misma variable contra varios valores posibles, evitando múltiples if...else anidados; cada caso debe cerrarse normalmente con break para no "caer" al siguiente caso. El operador ternario (condición ? valor1 : valor2) ofrece una forma compacta de escribir condicionales simples en una sola línea, muy usado para asignar valores de forma rápida.

Bucles: el bucle for es ideal cuando se conoce de antemano el número de iteraciones, ya que combina inicialización, condición e incremento en una sola línea. El bucle while repite un bloque mientras la condición sea verdadera, útil cuando no se sabe cuántas veces se repetirá. El bucle do...while es similar, pero garantiza que el bloque se ejecute al menos una vez antes de evaluar la condición. El bucle for...of recorre directamente los valores de estructuras iterables como arrays, strings, Maps o Sets, mientras que for...in recorre las claves (propiedades) enumerables de un objeto. Las palabras clave break y continue permiten controlar el flujo dentro de los bucles: break detiene por completo el bucle, mientras que continue salta a la siguiente iteración sin ejecutar el resto del código de la iteración actual.

3. Tipos de datos

Tipos primitivos (inmutables, se almacenan y se copian por valor): Number (representa tanto enteros como decimales), String (cadenas de texto, delimitadas por comillas simples, dobles o plantillas de texto con backticks), Boolean (true o false), Undefined (una variable declarada pero sin valor asignado), Null (representa la ausencia intencional de valor), BigInt (para enteros de precisión arbitraria, mayores al límite seguro de Number) y Symbol (valores únicos e inmutables usados normalmente como identificadores de propiedades).

Tipos por referencia (se almacenan en memoria y se copian por referencia, es decir, la variable guarda una dirección que apunta al mismo dato en memoria): Object, Array, Function y Date, entre otros. La

diferencia clave frente a los primitivos es que al asignar un objeto o array a otra variable, ambas variables apuntan al mismo dato en memoria, por lo que modificar una afecta también a la otra; en cambio, con los tipos primitivos cada variable mantiene una copia independiente de su valor. Esta distinción es fundamental para entender comportamientos comunes al comparar, copiar o pasar datos como argumentos de funciones.

4. Funciones

Declaradas (function declaration): se definen con la palabra reservada `function` seguida de un nombre, por ejemplo `function saludar() {...}`. Este tipo de función tiene "hoisting" completo, lo que significa que JavaScript la reconoce desde el inicio del archivo o del ámbito en el que se encuentra, permitiendo invocarla incluso antes de la línea donde fue escrita.

Expresadas (function expression): se crean asignando una función, con o sin nombre, a una variable, por ejemplo `const saludar = function() {...}`. A diferencia de las declaradas, no cuentan con hoisting completo (la variable existe pero no la función), por lo que deben definirse antes de ser utilizadas. Suelen emplearse como funciones anónimas pasadas como argumento a otras funciones.

Arrow functions (`=>`): introducidas en ES6, ofrecen una sintaxis más corta y clara, por ejemplo `const saludar = () => {...}`. Su característica más importante es que no poseen su propio valor de `this`, sino que heredan el del contexto léxico donde fueron definidas, lo que evita errores comunes al usar `this` dentro de callbacks. Por su brevedad, son muy utilizadas en métodos de arrays como `map`, `filter` o `forEach`.

5. Objetos y Arrays

Los objetos almacenan datos en pares clave-valor y permiten modelar entidades del mundo real con propiedades (datos) y métodos (funciones asociadas al objeto), por ejemplo: `{ nombre: "Ana", edad: 20, saludar() { return "Hola"; } }`. Se puede acceder a sus propiedades mediante notación de punto (`objeto.nombre`) o de corchetes (`objeto["nombre"]`), esta última útil cuando el nombre de la propiedad es dinámico. Los objetos son la base de la programación orientada a objetos en JavaScript, junto con conceptos como prototipos, herencia y las clases (`class`) introducidas en ES6 como una forma más clara de crear objetos y estructurar la herencia.

Los arrays son listas ordenadas de elementos, indexadas desde 0, que pueden almacenar cualquier tipo de dato, incluso mezclado (números, strings, objetos, otros arrays). Cuentan con métodos muy usados en el día a día: `push` y `pop` para agregar o quitar elementos al final, `shift` y `unshift` para hacerlo al inicio, `map` para transformar cada elemento y devolver un nuevo array, `filter` para obtener solo los elementos que cumplen una condición, `forEach` para recorrer sin retornar un nuevo array, y `reduce` para acumular los valores del array en un solo resultado. Estos métodos, combinados con arrow functions, permiten escribir código muy compacto y declarativo para trabajar con colecciones de datos.

6. Eventos y manipulación del DOM

El DOM (Document Object Model) representa la estructura HTML de una página como un árbol de nodos, donde cada etiqueta, atributo y texto se convierte en un objeto que JavaScript puede leer, modificar, eliminar o crear dinámicamente sin necesidad de recargar la página. Métodos como `document.getElementById`, `document.querySelector` y `document.querySelectorAll` permiten seleccionar uno o varios elementos según su id, clase o etiqueta; `createElement` y `appendChild` permiten crear nuevos elementos e insertarlos en el árbol; y propiedades como `innerHTML` o `textContent` permiten leer o modificar el contenido de un elemento, mientras que `classList` permite añadir, quitar o alternar clases CSS.

Los eventos son acciones que ocurren en la página, generadas por el usuario (`click`, `submit`, `keydown`, `mouseover`) o por el propio navegador (`load`, `resize`). El método `addEventListener` permite "escuchar"

estos eventos en un elemento y ejecutar una función (callback) en respuesta, sin sobrescribir otros manejadores de eventos que ya existan en ese mismo elemento. Dentro del manejador se recibe un objeto event con información sobre lo ocurrido (por ejemplo, event.target indica el elemento que disparó el evento), y métodos como preventDefault() permiten evitar el comportamiento por defecto del navegador, por ejemplo el envío automático de un formulario. Este mecanismo es la base para construir interfaces interactivas.

7. Asincronía

JavaScript necesita manejar tareas que no se resuelven de inmediato (peticiones a un servidor, lectura de archivos, temporizadores) sin bloquear el resto del programa, ya que se ejecuta en un solo hilo. Para esto existen distintos enfoques que han evolucionado con el tiempo:

Callbacks: son funciones que se pasan como argumento a otra función y se ejecutan una vez que la tarea asíncrona finaliza, por ejemplo al usar setTimeout o al leer datos. Fueron el primer mecanismo usado para manejar asincronía, pero cuando se necesitan varias operaciones encadenadas, los callbacks anidados dentro de otros callbacks generan un código difícil de leer y mantener, problema conocido como "callback hell" o "pyramid of doom".

Promesas (Promise): surgen como solución a los problemas de los callbacks. Una promesa representa un valor que estará disponible en el futuro (ya sea un éxito o un error), y puede encontrarse en tres estados: pending (pendiente, mientras se resuelve), fulfilled (cumplida, cuando la operación fue exitosa) y rejected (rechazada, cuando ocurrió un error). Se manejan encadenando .then() para procesar el resultado exitoso, .catch() para capturar errores y .finally() para ejecutar código sin importar el resultado, permitiendo encadenar varias operaciones asíncronas de forma más ordenada y legible que con callbacks anidados.

Async/await: es una sintaxis introducida en ES2017 construida sobre las promesas, que permite escribir código asíncrono con una apariencia sincrónica (secuencial), facilitando mucho su lectura. Se antepone la palabra async a la declaración de una función, lo cual hace que esta siempre retorne una promesa, y dentro de ella se usa await antes de una expresión que retorne una promesa, pausando la ejecución de esa función (sin bloquear el resto del programa) hasta que la promesa se resuelva. El manejo de errores en este caso suele hacerse con bloques try...catch, imitando el manejo de excepciones de la programación sincrónica tradicional.



JAVASCRIPT

